

Postgres Configuration and Performance Tuning

Hour 5 of PostgreSQL Training Day

SCaLE LA 2026

Hour 5 Topics

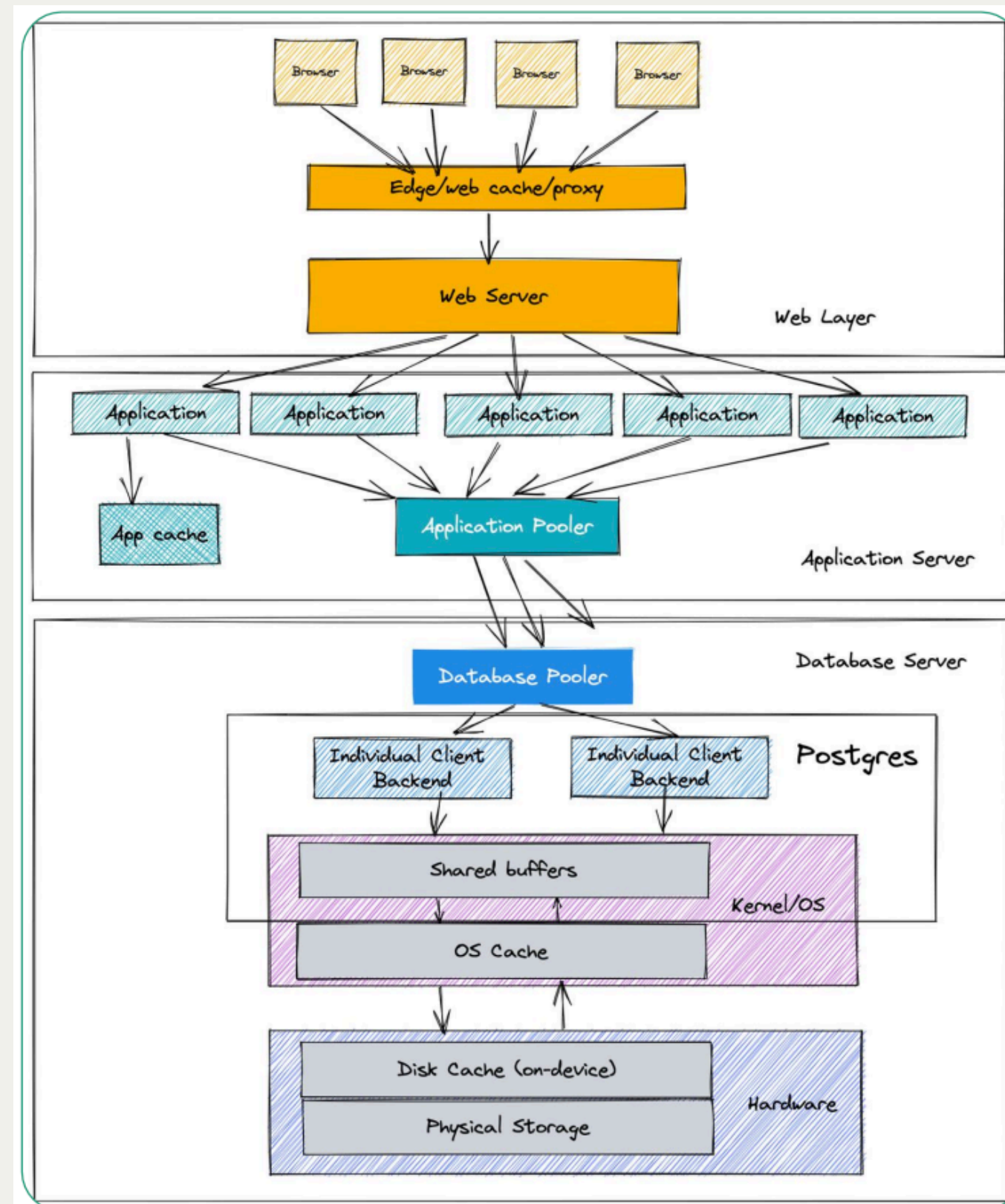
- Postgres architecture
- Memory configuration
- Cache hit ratio
- Shared buffers / Work memory
- I/O tuning
- Parallel query execution
- Vacuum and autovacuum

Training Materials

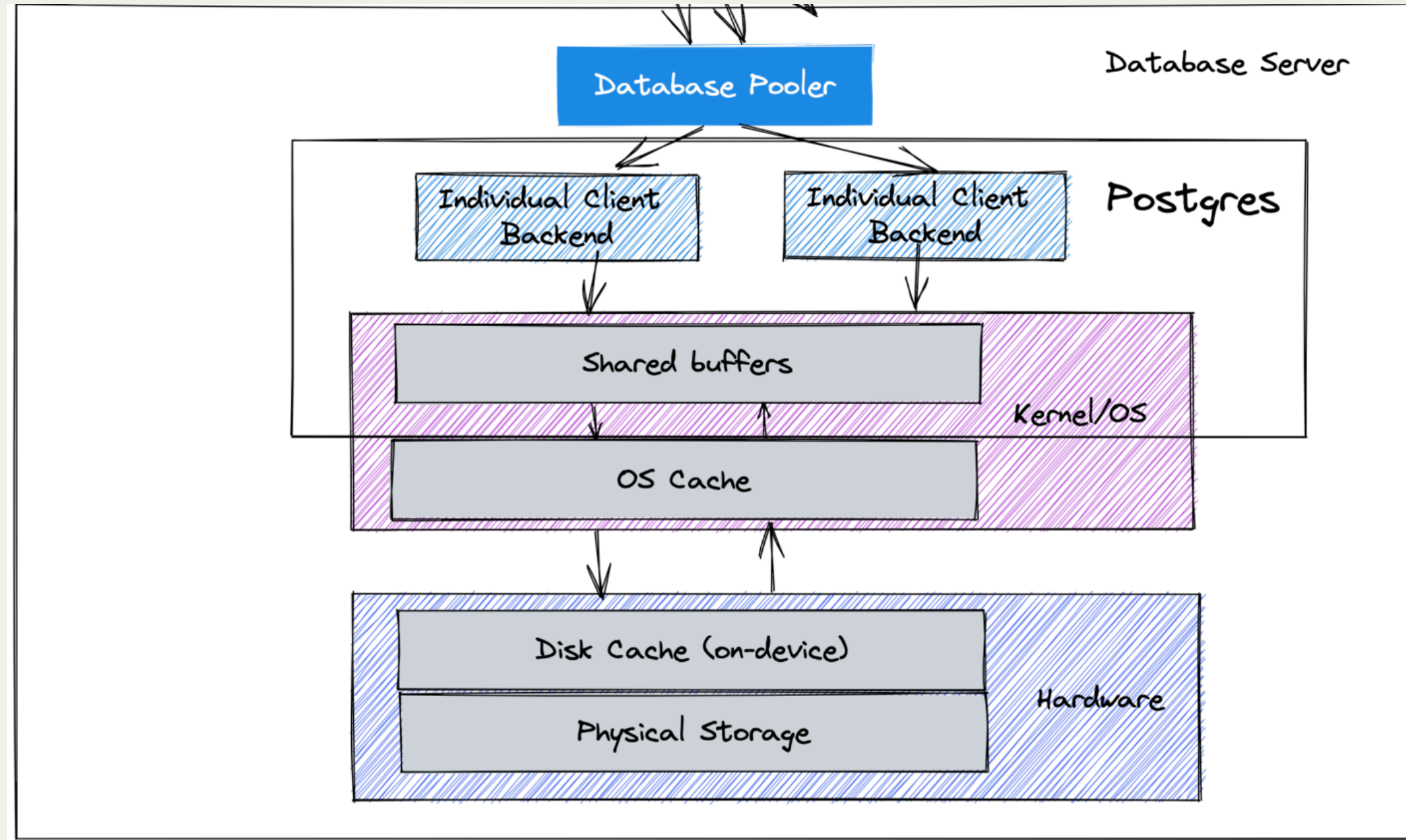
github.com/elizabeth-christensen/postgres-full-day-training



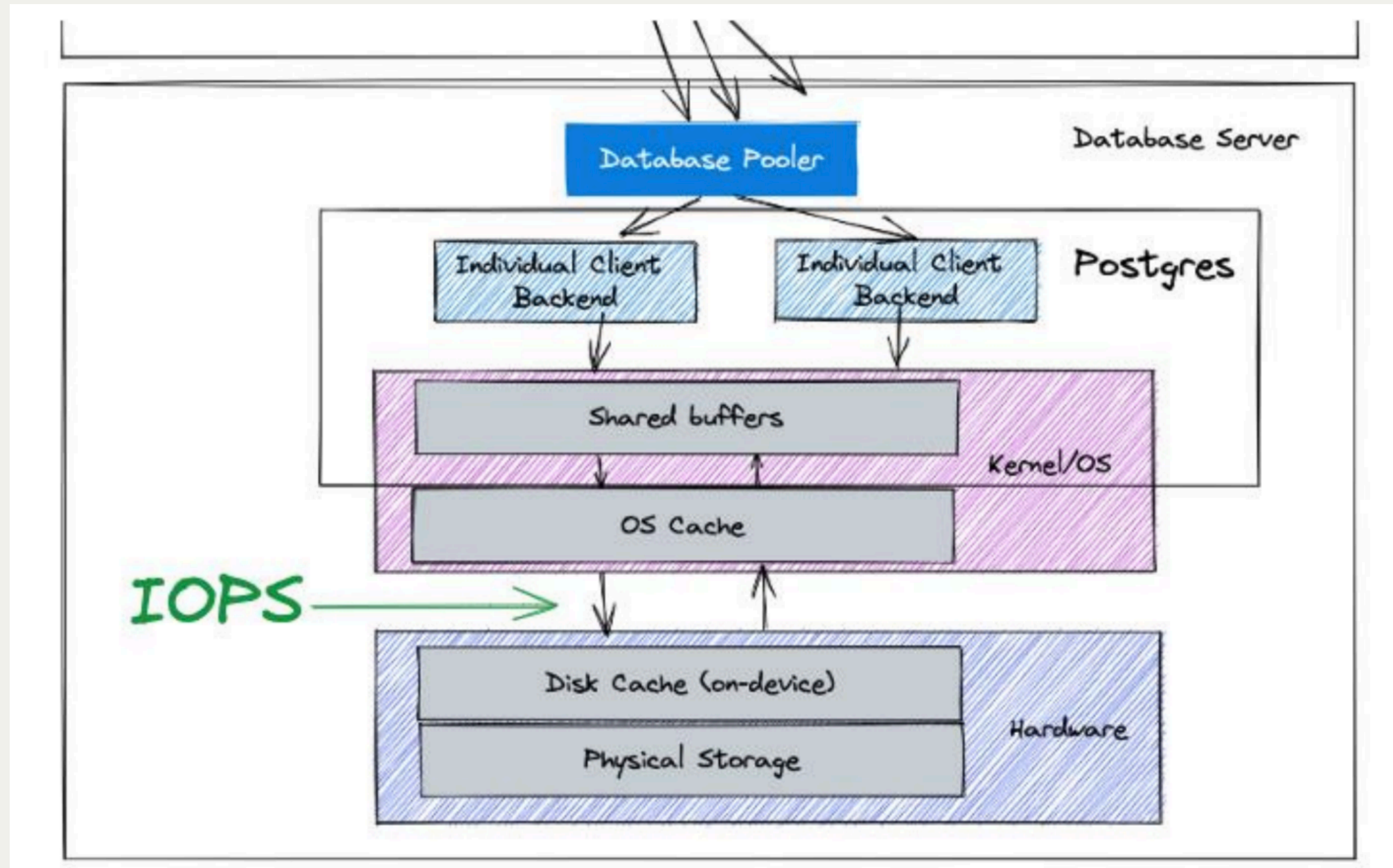
Postgres Architecture



Memory Architecture



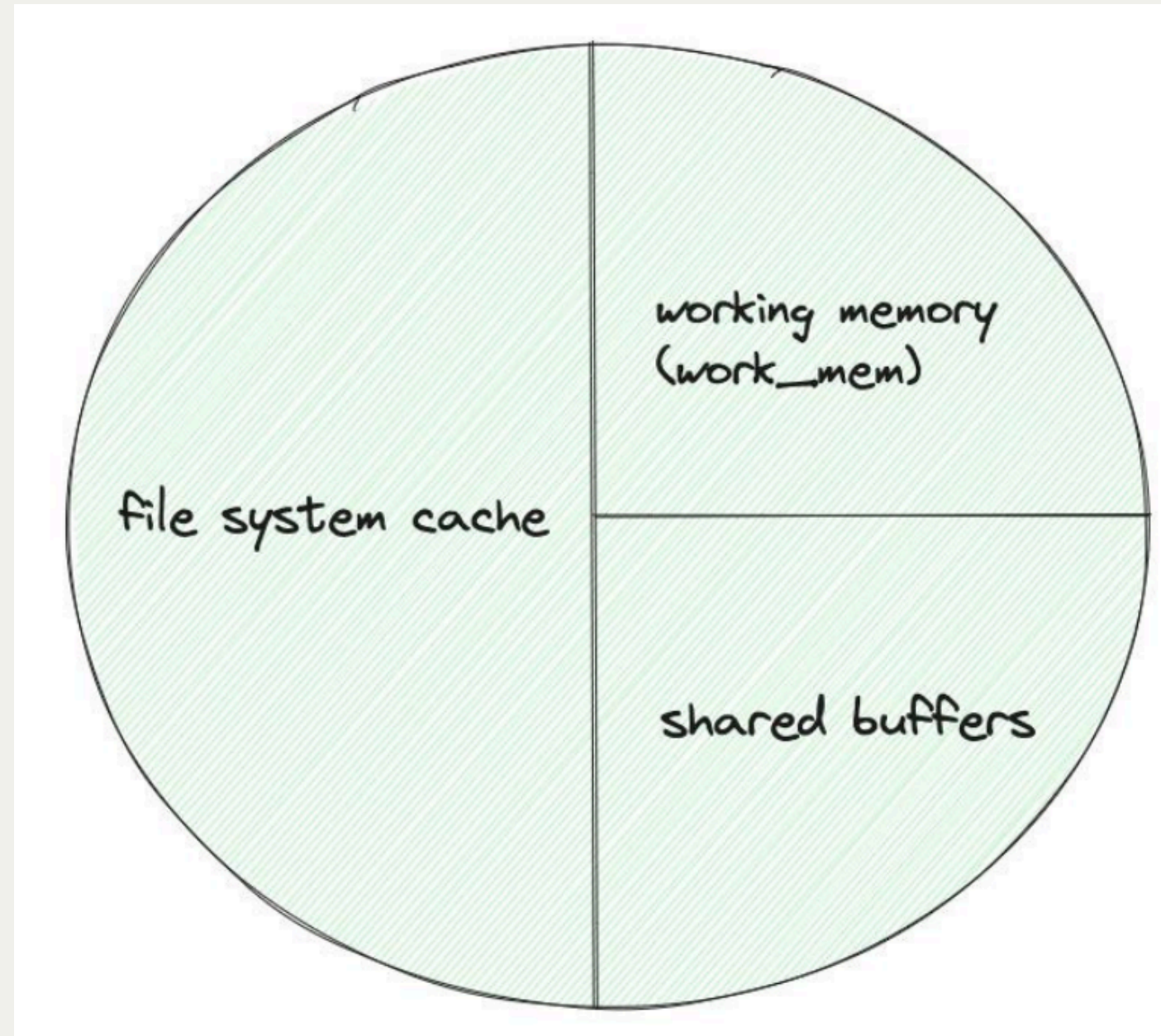
i/o



Background Processes

Process	Function
Background Writer	Writes dirty pages to disk
WAL Writer	Flushes WAL to disk
Checkpointer	Periodic full sync
Autovacuum	Cleans dead rows
Logger	Gathers logs
I/O worker	async I/O helper process
Archiver	Archives WAL files

Memory Configuration



Shared Buffers

The database page cache - most important setting

-- Check current value

`SHOW shared_buffers;`

-- Set in postgresql.conf

`shared_buffers = 8GB` -- For 32GB RAM system

-- Requires restart to change

Rule of thumb: Start with 25% of available RAM

Shared Buffers Guidelines

System RAM	shared_buffers
4GB	1GB
16GB	4GB
64GB	16GB
256GB	32-64GB

Beyond 32GB, diminishing returns - OS cache helps too

Effective Cache Size

Tells the planner how much memory is available for caching

- Set higher = planner prefers index scans
- Set lower = planner prefers sequential scans

`effective_cache_size = 16GB` -- For 32GB system

This is just a hint - doesn't allocate memory

Cache Hit Ratio

```
SELECT datname, blks_hit, blks_read,  
       ROUND(100.0 * blks_hit /  
             NULLIF(blks_hit + blks_read, 0), 2) as hit_ratio  
FROM pg_stat_database WHERE datname = 'bluebox';
```

datname	blks_hit	blks_read	hit_ratio
bluebox	109171275	301352	99.72

Target: > 99% for OLTP workloads ✓

What's in the Buffer Cache?

-- Install pg_buffercache extension

```
CREATE EXTENSION pg_buffercache;
```

-- See what's cached

```
SELECT
```

```
    c.relname,
```

```
    count(*) as buffers,
```

```
    pg_size_pretty(count(*) * 8192) as size
```

```
FROM pg_buffercache b
```

```
JOIN pg_class c ON b.relfilenode = c.relfilenode
```

```
WHERE b.reldatabase = (SELECT oid FROM pg_database WHERE datname = current_database())
```

```
GROUP BY c.relname
```

```
ORDER BY count(*) DESC
```

```
LIMIT 10;
```

Work Memory

What is work_mem?

Memory for query operations:

- Sorting (ORDER BY)
- Hash joins
- Hash aggregations
- Window functions

`SHOW work_mem;` -- Default: 4MB

work_mem Behavior

Each operation can use up to work_mem

-- This query might use 4 x work_mem

```
SELECT f.title, p.name, COUNT(*) as appearances
```

```
FROM film f
```

```
JOIN film_cast fc ON f.film_id = fc.film_id -- hash join
```

```
JOIN person p ON fc.person_id = p.person_id -- hash join
```

```
GROUP BY f.title, p.name -- hash aggregate
```

```
ORDER BY appearances DESC; -- sort
```

Setting work_mem

Be careful: $\text{work_mem} \times \text{connections} \times \text{operations}$

-- Global setting (conservative)

```
work_mem = 64MB
```

-- Increase for specific session

```
SET work_mem = '256MB';
```

-- Run complex analytical query

```
RESET work_mem;
```

When to Increase work_mem

Signs you need more:

-- Check for disk sorts

EXPLAIN (ANALYZE, BUFFERS) **SELECT** ...

-- Look for:

-- Sort Method: external merge ← disk sort, increase work_mem

-- Sort Method: quicksort ← memory sort, good!

Maintenance Work Memory

Used by maintenance operations:

```
maintenance_work_mem = 1GB
```

```
-- Affects:
```

```
-- VACUUM
```

```
-- CREATE INDEX
```

```
-- ALTER TABLE ADD FOREIGN KEY
```

Can be set much higher than work_mem

Checkpoints

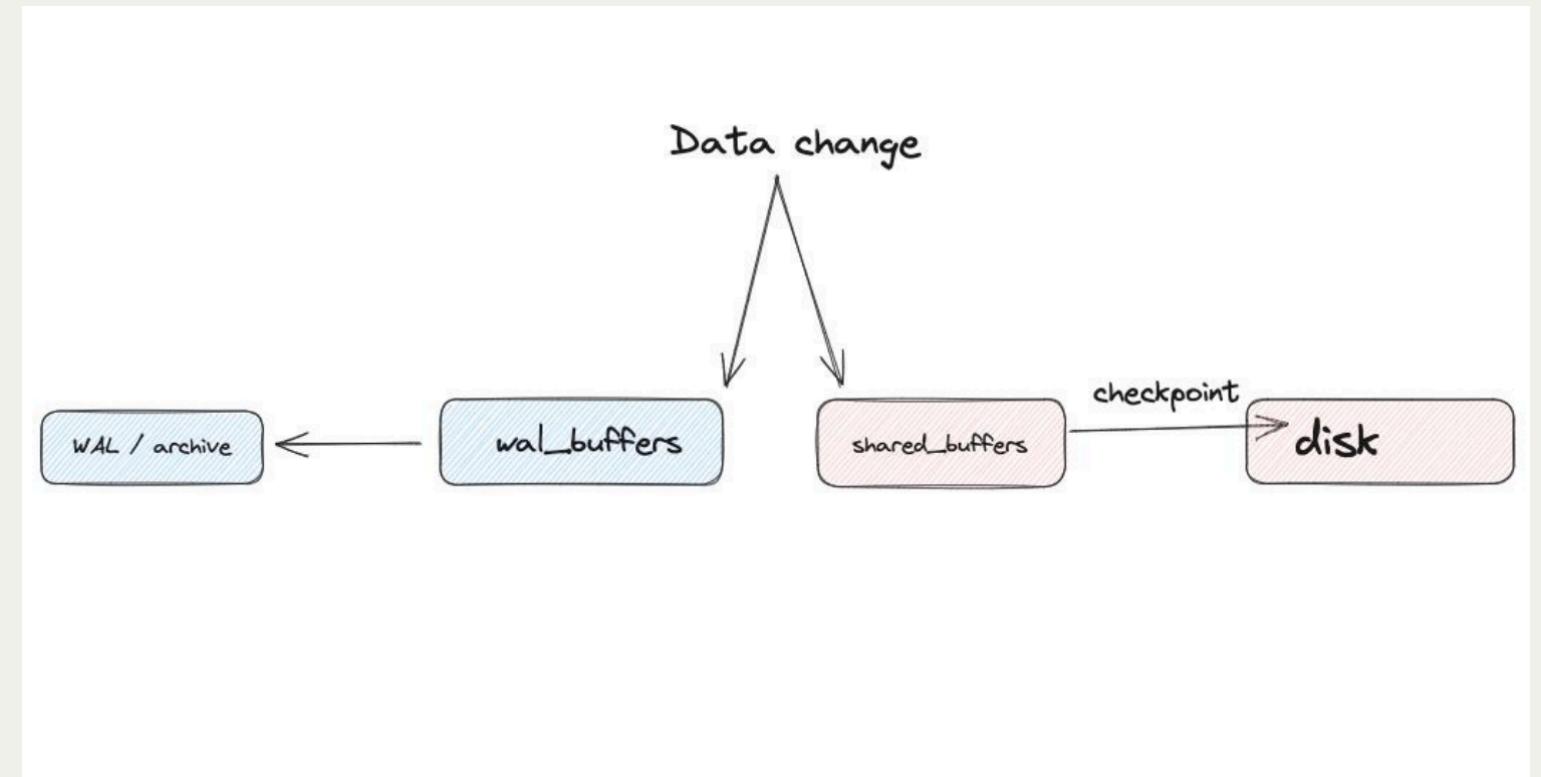
-- How often (in WAL segments or time)

`checkpoint_timeout = 15min`

`max_wal_size = 4GB`

-- How fast (spread I/O over time)

`checkpoint_completion_target = 0.9`



Checkpoint Tuning

More frequent checkpoints:

- + Faster crash recovery
- More I/O overhead
- More WAL written

Less frequent checkpoints:

- + Less I/O
- + Better performance
- Slower crash recovery
- Need more disk space for WAL

Sequential vs Random I/O Costs

-- Tells planner relative costs

seq_page_cost = 1.0 -- Sequential read cost

random_page_cost = 4.0 -- Random read cost (default)

-- For SSDs, reduce random_page_cost

random_page_cost = 1.1 -- SSDs have nearly equal random/sequential

Parallel Query Execution

Postgres can use multiple CPU cores for a single query:

- Sequential scans
- Hash joins
- Aggregations
- Index scans (Postgres 17+)

Parallel Query Defaults

-- Check current settings

SHOW max_parallel_workers_per_gather; -- Default: 2

SHOW max_parallel_workers; -- Default: 8

SHOW max_worker_processes; -- Default: 8

SHOW parallel_tuple_cost; -- Default: 0.1

SHOW min_parallel_table_scan_size; -- Default: 8MB

Parallel kicks in for larger tables automatically.

Increasing Parallel Workers

-- Allow more workers per query (requires restart for max_worker_processes)

```
ALTER SYSTEM SET max_worker_processes = 16;
```

```
ALTER SYSTEM SET max_parallel_workers = 12;
```

-- More workers per query operation

```
ALTER SYSTEM SET max_parallel_workers_per_gather = 4;
```

-- Apply changes

```
SELECT pg_reload_conf();
```

Rule of thumb: $\text{max_parallel_workers_per_gather} = \text{CPU cores} / 4$

Parallel Query in EXPLAIN

```
EXPLAIN ANALYZE SELECT count(*) FROM bluebox.film;
```

Finalize Aggregate

- > Gather

 - Workers Planned: 2

 - Workers Launched: 2

- > Partial Aggregate

 - > Parallel Seq Scan on film

"Workers Launched: 2" means parallel execution worked!

Memory Settings Summary

Setting	Purpose	Typical Value
shared_buffers	Database cache	25% of RAM
effective_cache_size	Planner hint	50-75% of RAM
work_mem	Sort/hash memory	16-256MB
maintenance_work_mem	VACUUM, CREATE INDEX	512MB-2GB

Vacuum and Autovacuum

Why Does Postgres Need Vacuum?

Postgres uses **MVCC** (Multi-Version Concurrency Control):

- UPDATE/DELETE doesn't erase rows immediately
- Old versions are marked as "dead tuples"
- Dead tuples = **bloat**

Vacuum reclaims this space for reuse.

Autovacuum: The Robot Cleaner

Autovacuum has been on by default since Postgres 8.3 (2008).

It triggers when:

Dead Tuples $>$ (Threshold + Scale Factor $\times$ Total Tuples)

Default: 50 rows + 20% of table size

For a 1,000 row table $\rightarrow$ vacuum at $\sim$ 250 dead rows

Do You Need to Tune Autovacuum?

Usually no! Defaults work for most workloads. But lots of dead rows can affect performance.

Consider tuning if you see:

- High bloat (> 50% of table size)
- Unusually high disk usage vs actual data
- Slow queries + high bloat
- I/O contention from vacuum running

Check Dead Tuples Waiting for Cleanup

```
SELECT
    relname AS table_name,
    n_dead_tup AS dead_rows,
    n_live_tup AS live_rows,
    last_autovacuum,
    last_vacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 10;
```

Tuning: Scale Factor

For large tables, 20% is too long to wait.

100M row table = waiting for 20M dead rows!

-- Vacuum more frequently (10% instead of 20%)

ALTER TABLE big_table

SET (autovacuum_vacuum_scale_factor = 0.1);

-- Or use absolute threshold for huge tables

ALTER TABLE huge_table

SET (autovacuum_vacuum_threshold = 10000000);

Tuning: Cost-Based Throttling

If vacuum is causing I/O pressure, slow it down:

-- Increase delay between vacuum operations (default 2ms)

```
ALTER TABLE busy_table
```

```
SET (autovacuum_vacuum_cost_delay = '5ms');
```

-- Lower cost limit = more frequent pauses

```
ALTER TABLE busy_table
```

```
SET (autovacuum_vacuum_cost_limit = 100);
```

Transaction ID Wraparound

Postgres uses 32-bit transaction IDs (~2 billion).

Vacuum "freezes" old rows so IDs can be reused.

If you hit wraparound → **database shuts down!**

-- Check oldest unfrozen transaction age

```
SELECT datname, age(datfrozenxid)
```

```
FROM pg_database
```

```
ORDER BY age DESC;
```

Stay well under 2 billion.

Vacuum Summary

Setting	When to Tune
<code>autovacuum_vacuum_scale_factor</code>	Large tables with high churn
<code>autovacuum_vacuum_cost_delay</code>	Vacuum causing I/O pressure
<code>autovacuum_freeze_max_age</code>	Anti-wraparound hitting peak hours

Rule of thumb: Aim for vacuum every ~2M dead rows on big tables.

Scaling Reads and Writes

Scaling Strategies

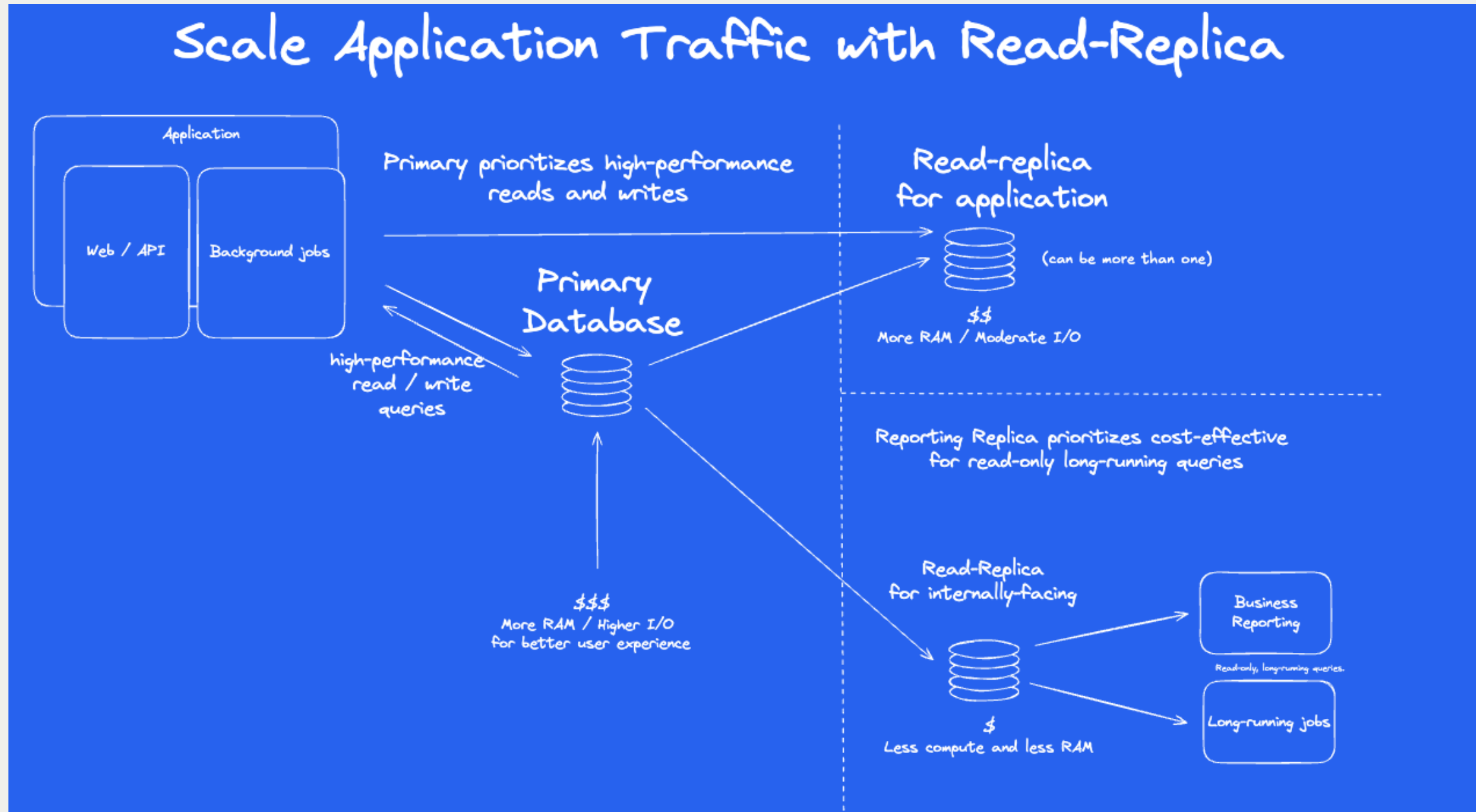
Vertical Scaling

- More CPU
- More RAM
- Faster storage
- Limits exist

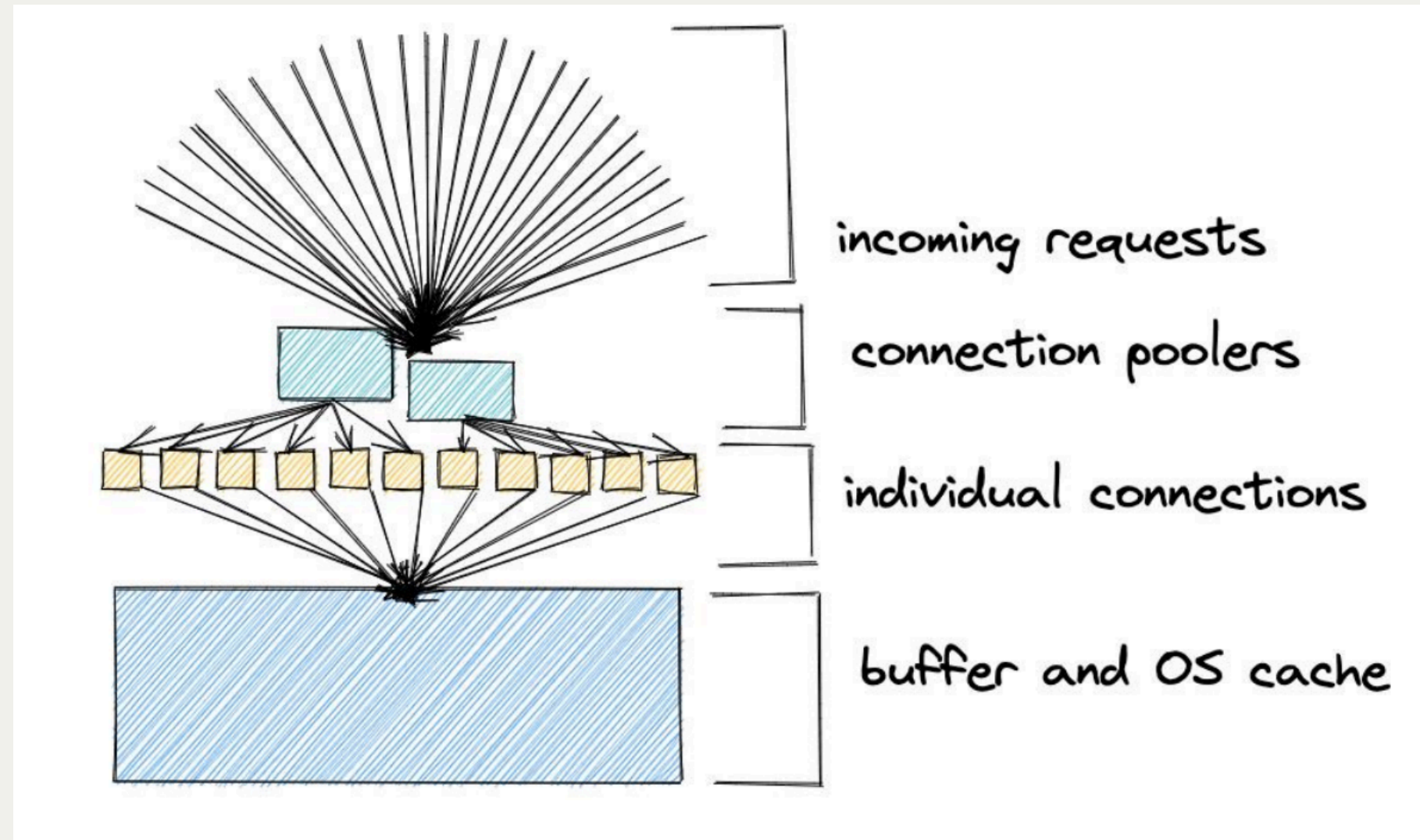
Horizontal Scaling

- Read replicas
- Connection pooling
- Partitioning
- Sharding

Read Scaling with Replicas



Connection Pooling



Reduces connection overhead dramatically

Table Partitioning

Break large tables into smaller pieces

-- Example: partitioning payment by date

```
CREATE TABLE bluebox.payment_partitioned (  
    payment_id SERIAL,  
    payment_date TIMESTAMPTZ NOT NULL,  
    customer_id INT,  
    amount NUMERIC(5,2)  
) PARTITION BY RANGE (payment_date);
```

```
CREATE TABLE payment_2024 PARTITION OF bluebox.payment_partitioned  
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

```
CREATE TABLE payment_2025 PARTITION OF bluebox.payment_partitioned  
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

Partitioning Benefits

- Faster queries with partition pruning
- Easier data archival (DROP old partitions)
- Parallel query per partition
- Smaller indexes per partition
- Faster VACUUM per partition

Write Scaling Challenges

PostgreSQL is single-writer by design

Options:

- Optimize single node (better hardware)
- Application-level sharding
- Citus extension (distributed tables)
- Accept eventual consistency with logical replication

Configuration Management

Where Settings Live

-- Show config file locations

`SHOW config_file;` -- Main: postgresql.conf

`SHOW hba_file;` -- Access: pg_hba.conf

`SHOW data_directory;` -- Data directory

Changing Settings

```
SELECT name, setting, unit, context
FROM pg_settings WHERE name IN
('shared_buffers', 'work_mem', 'effective_cache_size');
```

name	setting	unit	context
shared_buffers	1048576	8kB	postmaster
work_mem	262144	kB	user
effective_cache_size	524288	8kB	user

postmaster = restart, user = SET in session

Applying Changes

-- Reload configuration (no downtime)

```
SELECT pg_reload_conf();
```

-- Or from command line

```
pg_ctl reload -D /var/lib/postgresql/17/main
```

-- Check pending changes

```
SELECT name, setting, pending_restart
```

```
FROM pg_settings
```

```
WHERE pending_restart;
```

ALTER SYSTEM

-- Modify settings without editing file

```
ALTER SYSTEM SET work_mem = '128MB';
```

-- Written to postgresql.auto.conf

-- Takes effect after reload (or restart)

```
SELECT pg_reload_conf();
```

-- Reset to default

```
ALTER SYSTEM RESET work_mem;
```

Starting Point Configuration

-- For a dedicated 32GB database server:

shared_buffers = 8GB

effective_cache_size = 24GB

work_mem = 64MB

maintenance_work_mem = 2GB

max_connections = 200

max_wal_size = 4GB

checkpoint_timeout = 15min

random_page_cost = 1.1 -- SSD

effective_io_concurrency = 200 -- SSD

PGTune

Online tool for generating starting configurations

pgtune.leopard.in.ua

Input:

- PostgreSQL version
- OS type
- RAM
- CPU cores
- Storage type
- Workload type

Hour 5 Summary

-  Postgres architecture and processes
-  Memory: sharedbuffers, workmem
-  Cache hit ratio monitoring
-  I/O tuning: checkpoints, WAL
-  Parallel query configuration
-  Vacuum and autovacuum tuning
-  Scaling: replicas, pooling, partitioning
-  Configuration management best practices

Questions?

Next: Postgres Query Tuning